

CASTAGNET

Export ONNX et évaluation embarquée

Rapport final - §4.3

Mise en Situation Professionnelle - MSc BIHAR
4 septembre 2026

Dorian ARGAILLOT

Configuration figée

ResNet18 • checkpoint epoch 13 • ONNX opset 17

Machine mesurée : RTX 3080 Laptop 8 Go - cible : GTX 1060 3 Go

Synthèse décisionnelle

Réponse à la question du §4.3 - ResNet18 est raisonnablement exécutable avec ONNX Runtime sur la machine de développement. Son inférence CUDA est rapide, mais la faisabilité sur la GTX 1060 3 Go et les 12 flux réels n'est pas démontrée. Le goulot observé est principalement la préparation CPU des images.

Le choix prédictif du §4.2 reste cohérent pour un déploiement GPU : ResNet18 combine la meilleure macro-F1 de validation (85,21 %) et la meilleure latence CUDA batch 1 mesurée parmi les trois architectures. En revanche, la chaîne complète atteint 52,47 images/s en batch 1 et au mieux 58,04 images/s avec batching, soit légèrement moins que les 60 images/s correspondant au scénario de douze caméras analysées à 5 FPS chacune.

- Mesure réelle : Windows 11, CPU Intel64 Family 6 Model 165, 16 CPU logiques, RTX 3080 Laptop 8 Go.
- Cible non mesurée : GTX 1060 3 Go, ancien Intel i7 et douze flux caméra.
- Résultat prédictif figé du §4.2 : précision Conforme 89,16 %, rappel Conforme 85,54 %, macro-F1 84,44 % sur test avec seuil 0,55.
- Priorité future : optimiser et paralléliser le décodage et le preprocessing avant de chercher uniquement un réseau plus petit.

Sommaire

1. Export ONNX et protocole de benchmark.....	4
1.1 Objectifs et contraintes matérielles	4
1.2 Export ONNX de ResNet18	4
1.3 Validation numérique PyTorch / ONNX.....	5
1.4 Protocole de benchmark	6
2. Performances et comparaison des modèles	6
2.1 Performances CPU et CUDA de ResNet18.....	6
2.2 Comparaison ResNet18 / EfficientNet / MobileNet.....	8
2.3 Analyse du compromis qualité / coût	9
3. Étude du déploiement et des 12 flux	11
3.1 Étude du batch dynamique	11
3.2 Analyse de la capacité 12 flux	12
3.3 Identification des goulots d'étranglement	14
3.4 Discussion de la cible GTX 1060 3 Go	14
3.5 Limites et recul critique.....	16
3.6 Pistes d'optimisation	16
3.7 Conclusion du §4.3	17

1. Export ONNX et protocole de benchmark

1.1 Objectifs et contraintes matérielles

Le §4.3 évalue si le modèle retenu au §4.2 peut être intégré dans une chaîne d'inférence embarquée. Il ne cherche ni à modifier les poids ni à améliorer les métriques de classification. L'étude porte sur l'export, la fidélité numérique, les temps CPU/GPU, le coût end-to-end et le potentiel de mutualisation pour douze caméras.

Tableau 1 - Configuration figée héritée du §4.2

Élément	Valeur	Justification
Modèle	ResNet18	Meilleur compromis prédictif mesuré au §4.2
Checkpoint	Epoch 13	Meilleure loss de validation du run retenu
Run MLflow	db0ee7873903411e899e2f85384ae58a	Traçabilité des poids
Entrée / sortie	RGB 224 × 224 / 4 logits	Conforme au preprocessing et au mapping entraînés
Mapping	Conforme 0 ; NON Conforme 1 ; PIETRA 2 ; Vide 3	Ordre figé des classes
Seuil métier	P(Conforme) = 0,55, hors graphe	Règle figée sur validation et indépendante du réseau

Source : config.json, export_summary.json et final_test_metrics.json

Cahier des charges prédictif - Sur le test final déjà ouvert au §4.2, le rappel Conforme atteint 85,54 %, mais la précision de 89,16 % reste sous l'objectif de 95 %. Le §4.3 conserve ce résultat sans ajustement.

Tableau 2 - Machine mesurée et matériel cible

Statut	CPU / système	GPU	Portée
Mesure réelle	Windows 11 ; 16 CPU logiques Intel64 Family 6 Model 165	RTX 3080 Laptop 8 192 Mio ; pilote 591.74	Tous les chiffres de latence du rapport
Cible du sujet	Ancien Intel i7	GTX 1060 3 Go	Analyse qualitative uniquement ; aucune extrapolation

Source : benchmark_results.json

Débit matière - Le cahier des charges mentionne également un débit de 100 kg/h. Les mesures réalisées ici portent sur un nombre d'images traitées par seconde et ne permettent pas, à elles seules, de convertir ce débit matière en cadence d'inférence nécessaire. Cette conversion dépend notamment du nombre de fruits visibles simultanément, de leur vitesse de déplacement et de leur durée de présence dans le champ des caméras. Les scénarios de 1, 2, 5, 10 et 25 FPS par caméra sont donc utilisés comme scénarios de sensibilité, et non comme traduction directe des 100 kg/h.

1.2 Export ONNX de ResNet18

ONNX découple le réseau du framework PyTorch utilisé pendant l'entraînement et permet son exécution avec ONNX Runtime sur CPU ou GPU. L'opset 17 couvre les opérateurs de ResNet18 et est compatible avec la version du runtime utilisée ; des opérateurs plus récents n'étaient pas nécessaires.

Tableau 3 - Caractéristiques de l'export ONNX principal

Propriété	Valeur	Contrôle
Fichier	resnet18_castagnet.onnx	Export batch fixe conservé
Entrée	input : [1, 3, 224, 224]	Batch 1, RGB
Sortie	logits : [1, 4]	Quatre classes

Propriété	Valeur	Contrôle
Opset	17	Validé par ONNX checker
Taille	44 704 751 octets 42,63 Mio	Poids FP32 exportés
Softmax	Absent	Le graphe expose les logits
Seuil 0,55	Absent	Post-traitement métier externe
SHA-256	e663bf4b6f3746072ea978f53ba6d958f3ea49c591c5a97311bebea219bcfd23	Intégrité de l'artefact

Source : `export_summary.json` et modèle ONNX

Le Softmax et le seuil ne sont pas intégrés : le réseau reste un classifieur générique produisant des logits, tandis que la règle métier demeure explicite et modifiable indépendamment du modèle. Cette séparation évite de confondre calcul neuronal et décision opérationnelle.

1.3 Validation numérique PyTorch / ONNX

La fidélité a été contrôlée sur 16 images déterministes, régulièrement espacées dans le manifeste de validation. Ce choix rend le contrôle reproductible sans rouvrir le test final. Les sorties ne sont pas bit-à-bit identiques, mais les écarts FP32 sont de très faible amplitude.

Tableau 4 - Fidélité de l'export ResNet18 batch 1

Indicateur	Valeur	Interprétation
Max. écart absolu logits	1.6212463e-05	Écart numérique très faible
Écart absolu moyen logits	4.3087639e-06	Très faible
Max. écart probabilités	2.0861626e-06	Très faible
Écart moyen probabilités	1.9503277e-07	Très faible
Argmax différents	0	Aucune classe modifiée
Décisions seuil 0,55 différentes	0	Aucune décision métier modifiée

Source : `export_summary.json` et `numerical_validation_details.csv`

Verdict - L'export ONNX est fidèle à PyTorch sur l'échantillon contrôlé et peut être utilisé pour les mesures d'inférence.

1.4 Protocole de benchmark

Tableau 5 - Choix méthodologiques et justification

Choix	Application	Pourquoi dans CastagNet ?
Batch 1	Référence embarquée	Représente la décision immédiate sur une image
50 warmups	Avant chaque série	Écarte les coûts d'initialisation non représentatifs du régime stabilisé
500 itérations	Benchmark statique	Réduit la sensibilité aux variations ponctuelles
Moyenne, médiane, p95, p99	Distribution des latences	Les percentiles montrent les ralentissements que la moyenne masque
Inference-only / end-to-end	Mesures séparées	Distingue le coût du réseau du coût applicatif réel
CPU / CUDA	Deux providers	Compare repli CPU et accélération GPU sur la machine disponible
TensorRT non retenu	Provider détecté seulement	Aucun benchmark final TensorRT n'a été produit

Source : `benchmark_onnx.py` et `benchmark_results.json`

Les providers détectés sont `TensorrtExecutionProvider`, `CUDAExecutionProvider`, `CPUExecutionProvider`. Les références utilisent `CPUExecutionProvider` et `CUDAExecutionProvider`, qui fournissent une comparaison directe et reproductible. Le temps de création de session est exclu ; le décodage est inclus dans le preprocessing.

2. Performances et comparaison des modèles

2.1 Performances CPU et CUDA de ResNet18

Tableau 6 - Benchmark ResNet18 ONNX batch 1

Provider	Périmètre	Moy. ms	Méd. ms	p90	p95	p99	img/s
CPU	Inférence seule	17,756	17,539	18,984	20,213	22,202	56,32
CPU	End-to-end	36,176	32,082	48,349	53,934	76,223	27,64
CUDA	Inférence seule	3,865	3,661	4,546	4,970	5,892	258,71
CUDA	End-to-end	19,057	18,809	22,613	24,126	28,737	52,47

Source : benchmark_results.json - 50 warmups, 500 itérations

Sur CUDA, l'inférence seule prend 3,865 ms, contre 17,756 ms sur CPU. Le pipeline CUDA complet atteint toutefois 19,057 ms : le preprocessing seul représente déjà 10,657 ms en moyenne. La RAM maximale observée est de 653,83 Mio sur CPU et 1109,75 Mio pendant la série CUDA ; la VRAM du processus n'a pas été obtenue.

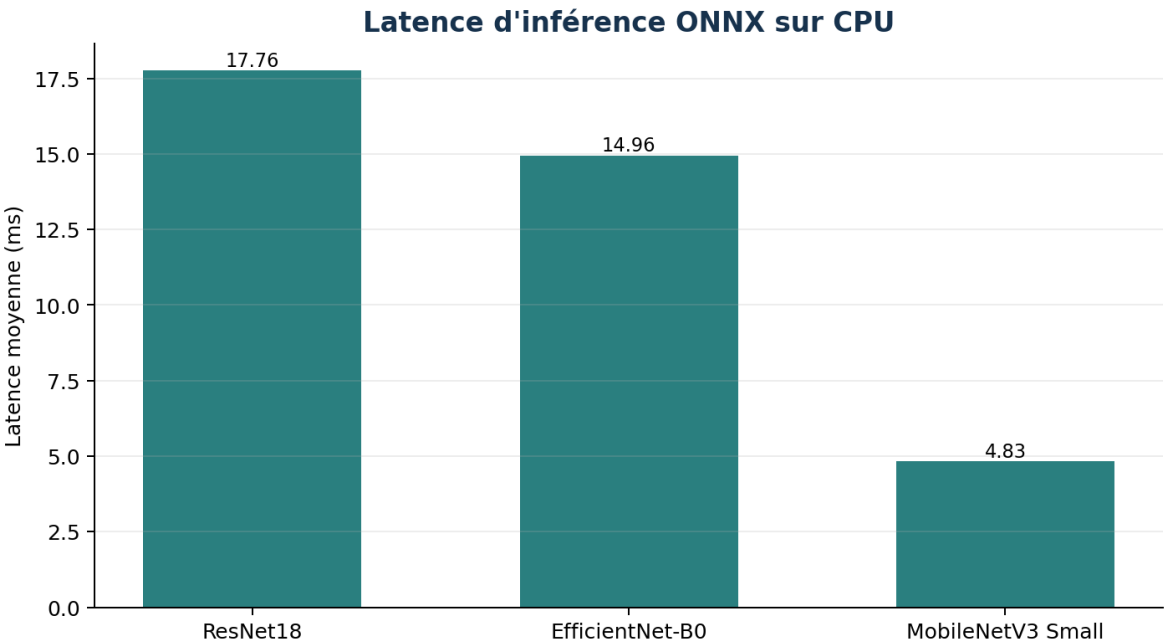


Figure 1 - Latence d'inférence CPU des trois modèles. Source : onnx_comparison.csv

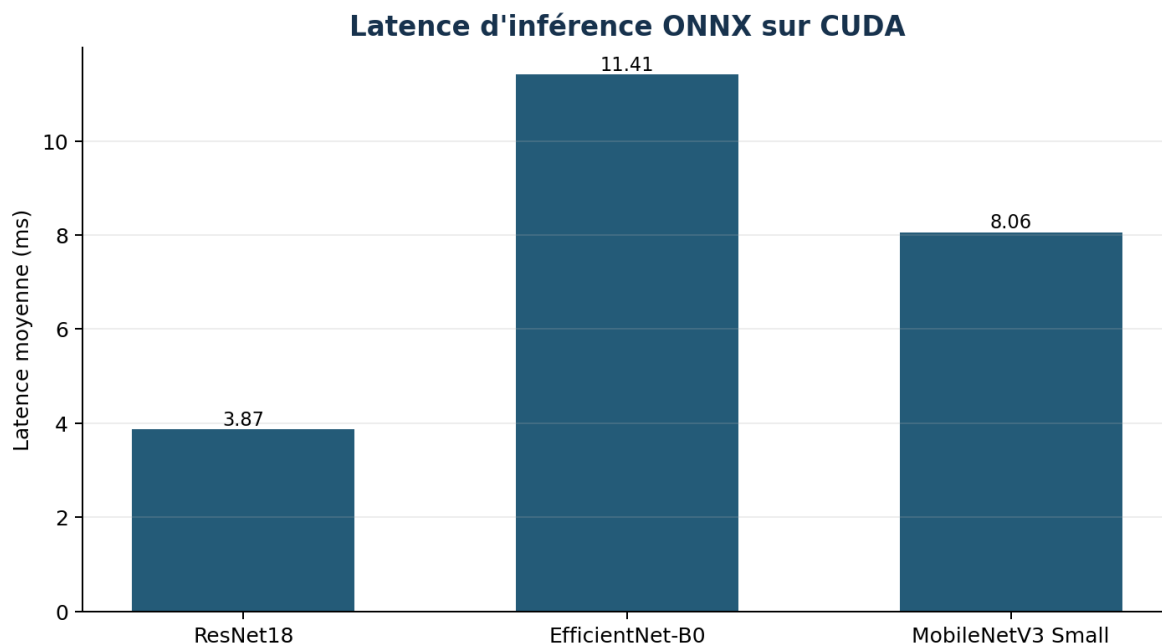


Figure 2 - Latence d'inférence CUDA des trois modèles. Source : onnx_comparison.csv

Les mesures séparées ne doivent pas être additionnées pour reconstruire l'end-to-end : les boucles, allocations et états de cache diffèrent. La mesure combinée peut inclure décodage, conversions, constitution des tenseurs, copies mémoire, orchestration Python et synchronisations CUDA. Faute d'instrumentation plus fine dans ce benchmark, aucune cause unique n'est attribuée. L'end-to-end reste la référence applicative.

2.2 Comparaison ResNet18 / EfficientNet / MobileNet

Les trois architectures effectivement entraînées au §4.2 ont été exportées avec le même format afin de vérifier si un réseau plus léger procure un avantage réel. La taille et le nombre de paramètres ne suffisent pas à anticiper la latence d'un runtime donné.

Tableau 7 - Qualité de validation et coût ONNX

Modèle	Macro-F1	Paramètres	ONNX Mio	CPU inf.	CUDA inf.	CPU E2E	CUDA E2E
ResNet18	85,21 %	11 178 564	42,63	17,756 ms 56,32 img/s	3,865 ms 258,71 img/s	36,176 ms	19,057 ms
EfficientNet-B0	83,60 %	4 012 672	15,30	14,956 ms 66,86 img/s	11,408 ms 87,66 img/s	32,278 ms	24,416 ms
MobileNetV3 Small	80,49 %	1 521 956	5,81	4,834 ms 206,87 img/s	8,061 ms 124,06 img/s	21,533 ms	19,315 ms

Source : onnx_comparison.csv ; métriques validation figées du §4.2

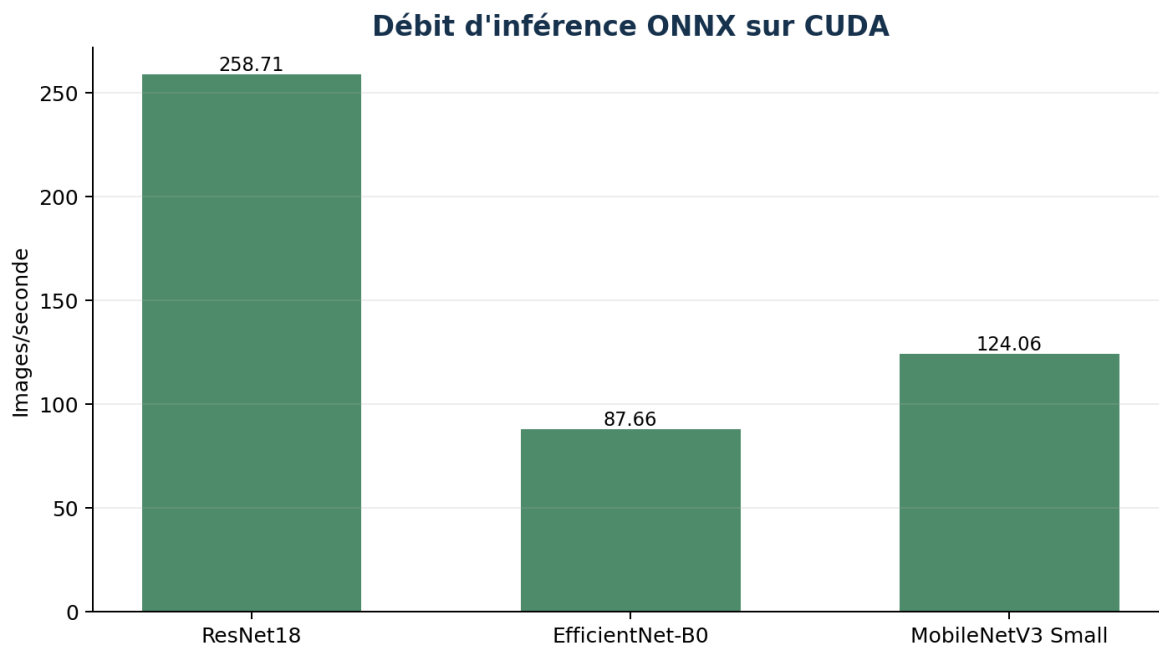


Figure 3 - Débit d'inférence CUDA des trois architectures. Source : `onnx_comparison.csv`

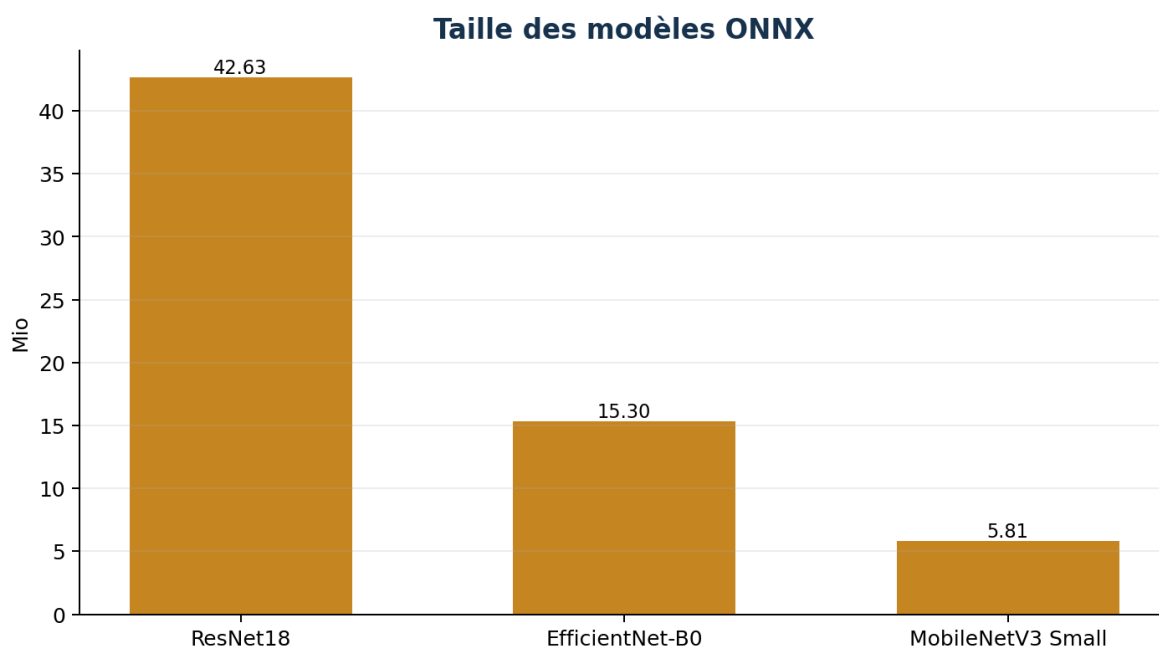


Figure 4 - Taille des trois modèles ONNX. Source : `export_summary.json` des trois architectures

2.3 Analyse du compromis qualité / coût

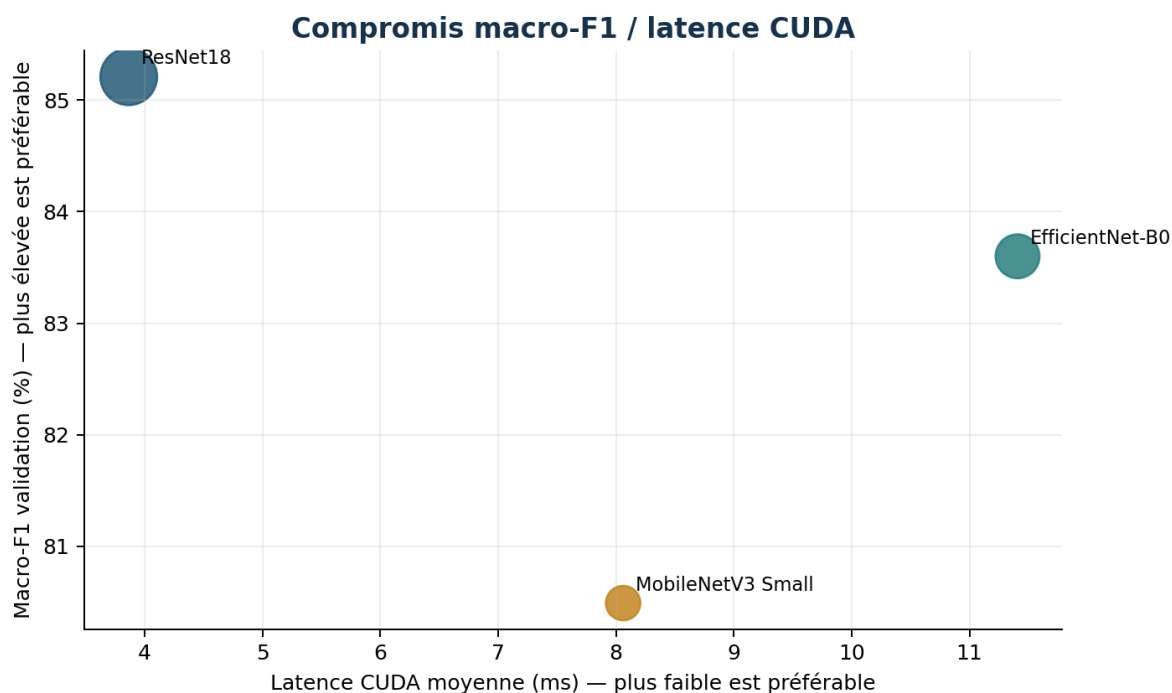


Figure 5 - Macro-F1 de validation et latence CUDA batch 1. Source : onnx_comparison.csv

Résultat CUDA - ResNet18 est à la fois le meilleur prédictivement, le plus rapide en inférence seule et légèrement le plus rapide end-to-end sur la RTX 3080 mesurée. Un modèle plus petit n'est donc pas automatiquement plus rapide : l'efficacité dépend aussi des opérateurs, de leur parallélisation et du runtime.

Sur CPU, MobileNetV3 Small est nettement plus attractif : 4,834 ms par image contre 17,756 ms pour ResNet18, avec un fichier de 5,81 Mio. Ce gain s'accompagne d'une macro-F1 inférieure de 4,72 points. Le choix dépend donc du contexte : ResNet18 reste cohérent pour le GPU mesuré, tandis que MobileNet constitue une option crédible si le déploiement devient principalement CPU ou très contraint en stockage.

Tableau 8 - Fidélité ONNX des modèles alternatifs

Modèle	Max. écart logits	Écart moyen logits	Max. écart probabilités	Argmax différents	Verdict
EfficientNet-B0	0,000931740	0,000149926	0,000073820	0	Valide
MobileNetV3 Small	0,000210762	0,000039052	0,000024498	0	Valide

Source : export_summary.json de chaque architecture

3. Étude du déploiement et des 12 flux

3.1 Étude du batch dynamique

Avec douze caméras, plusieurs images peuvent arriver dans une fenêtre proche. Un second export autorise donc N images par appel afin d'étudier le potentiel de mutualisation GPU. Il conserve exactement le checkpoint epoch 13, l'opset 17, les quatre logits et le post-traitement externe ; il ne remplace pas l'export batch 1.

Tableau 9 - Validation numérique de l'export dynamique

Batch	Max. écart logits	Max. écart probabilités	Argmax différents	Décisions 0,55 différentes
1	0.000008106	0.000000018	0	0
4	0.000008106	0.000000268	0	0
12	0.000017166	0.000002041	0	0

Source : dynamic_export_summary.json - validation uniquement

Tableau 10 - Benchmark CUDA du batch dynamique

Batch	Inf. ms/batch	Inf. p95	Inf. ms/image	Inf. img/s	E2E ms/batch	E2E ms/image	E2E img/s
1	4,115	5,084	4,115	243,02	20,473	20,473	48,85
2	4,976	6,802	2,488	401,95	51,727	25,864	38,66
4	5,806	8,748	1,452	688,93	90,512	22,628	44,19
8	6,250	7,660	0,781	1280,04	137,826	17,228	58,04
12	7,854	8,369	0,654	1527,94	237,794	19,816	50,46

Source : batch_benchmark.json - 50 warmups, 300 itérations par taille

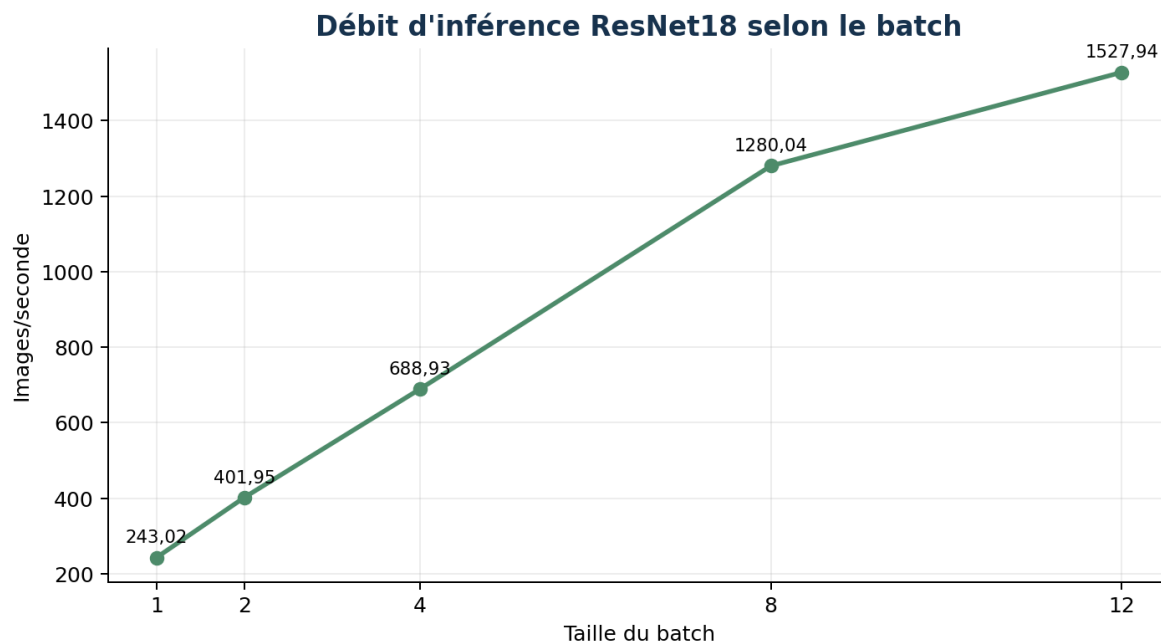


Figure 6 - Débit d'inférence ResNet18 selon la taille du batch. Source : batch_benchmark.json

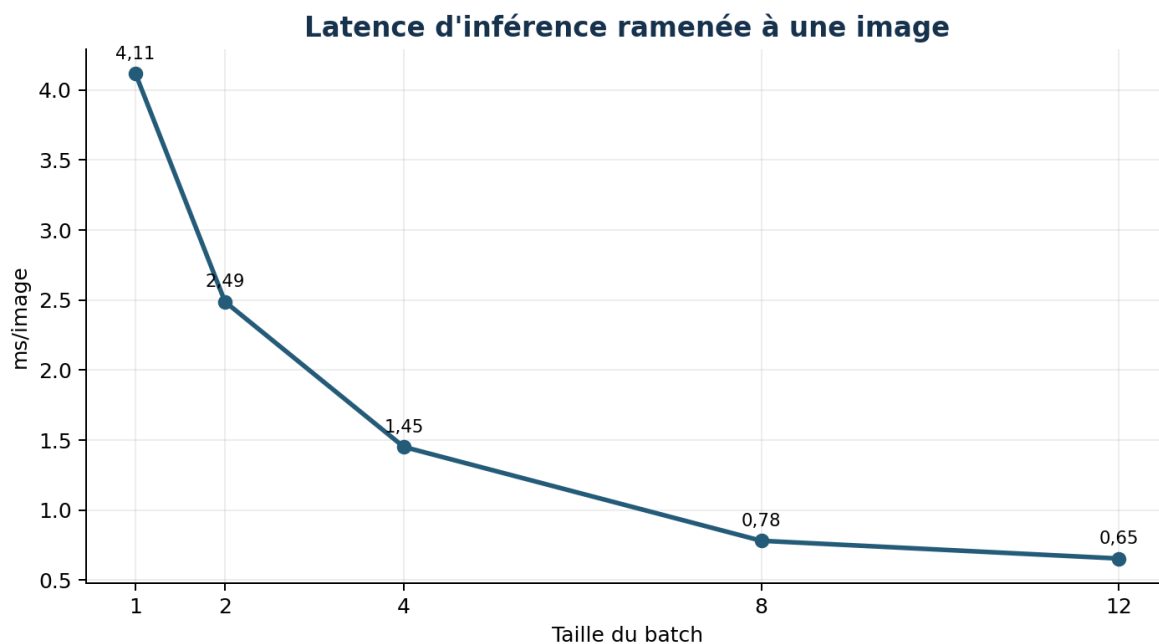


Figure 7 - Latence d'inférence ramenée à une image. Source : batch_benchmark.json

Entre les batches 1 et 12, le débit d'inférence passe de 243,02 à 1 527,94 images/s, soit environ $\times 6,3$, et la latence par image de 4,115 à 0,654 ms. Le batch 12 simule uniquement la possibilité de regrouper douze images dans un appel d'inférence ; il ne reproduit ni l'arrivée simultanée de douze flux vidéo, ni leur décodage concurrent.

3.2 Analyse de la capacité 12 flux

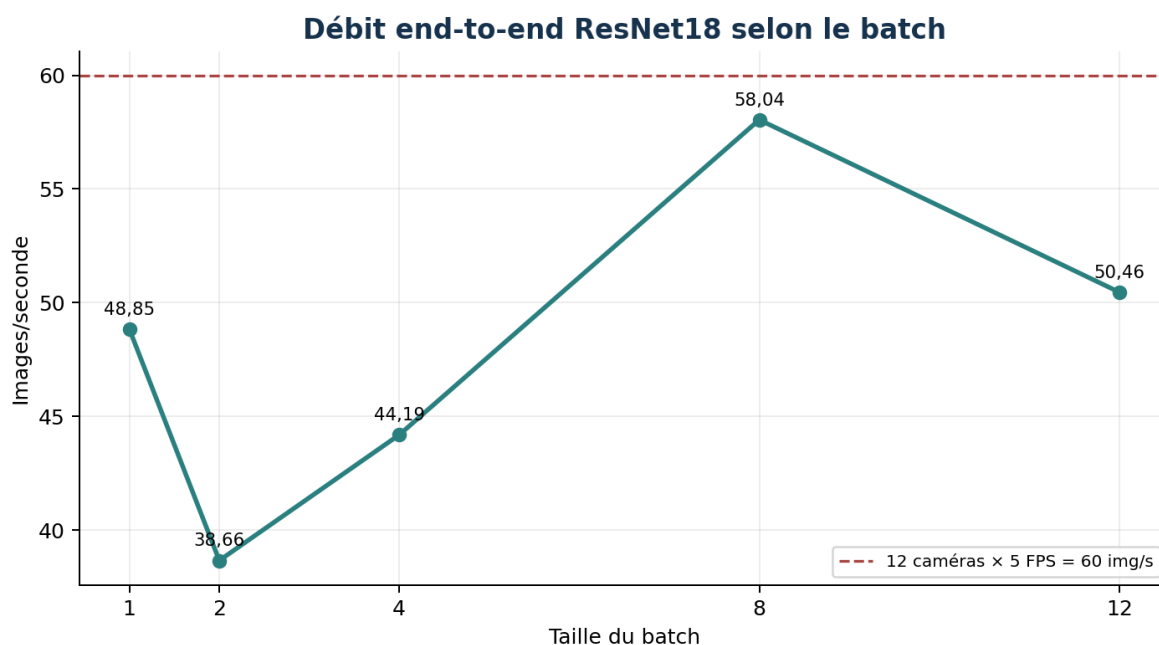


Figure 8 - Débit end-to-end selon le batch et scénario à 5 FPS/caméra. Source : batch_benchmark.json

Tableau 11 - Scénarios théoriques associés aux douze caméras

Cadence/caméra	Débit correspondant	Meilleur E2E mesuré	Couverture sur machine mesurée
1 FPS	12 img/s	58,04 img/s	Couverte théoriquement
2 FPS	24 img/s	58,04 img/s	Couverte théoriquement
5 FPS	60 img/s	58,04 img/s	Proche mais non couverte
10 FPS	120 img/s	58,04 img/s	Non couverte

Cadence/caméra	Débit correspondant	Meilleur E2E mesuré	Couverture sur machine mesurée
25 FPS	300 img/s	58,04 img/s	Non couverte

Source : batch_benchmark.json - calcul séquentiel simplifié

Lecture correcte - Le scénario 5 FPS par caméra est proche de la capacité mesurée mais reste légèrement supérieur au meilleur débit end-to-end observé : 60 img/s correspondent à ce scénario contre 58,04 img/s mesurés. Il n'est donc pas atteint.

Le contraste est déterminant : l'inférence seule atteint 1 527,94 images/s au batch 12, alors que le meilleur pipeline complet atteint 58,04 images/s au batch 8. La performance du réseau seul ne représente donc pas la performance applicative.

3.3 Identification des goulots d'étranglement

Tableau 12 - Mesures isolées au batch 12

Étape	Moyenne ms/batch	Périmètre
Décodage	61,586	Ouverture PIL, décodage, conversion RGB
Preprocessing hors décodage	80,368	Resize, padding, tenseur et normalisation
Constitution du batch	5,858	Empilement NumPy
session.run	7,854	Copie hôte→GPU incluse, CUDA, synchronisation et sortie

Source : batch_benchmark.json

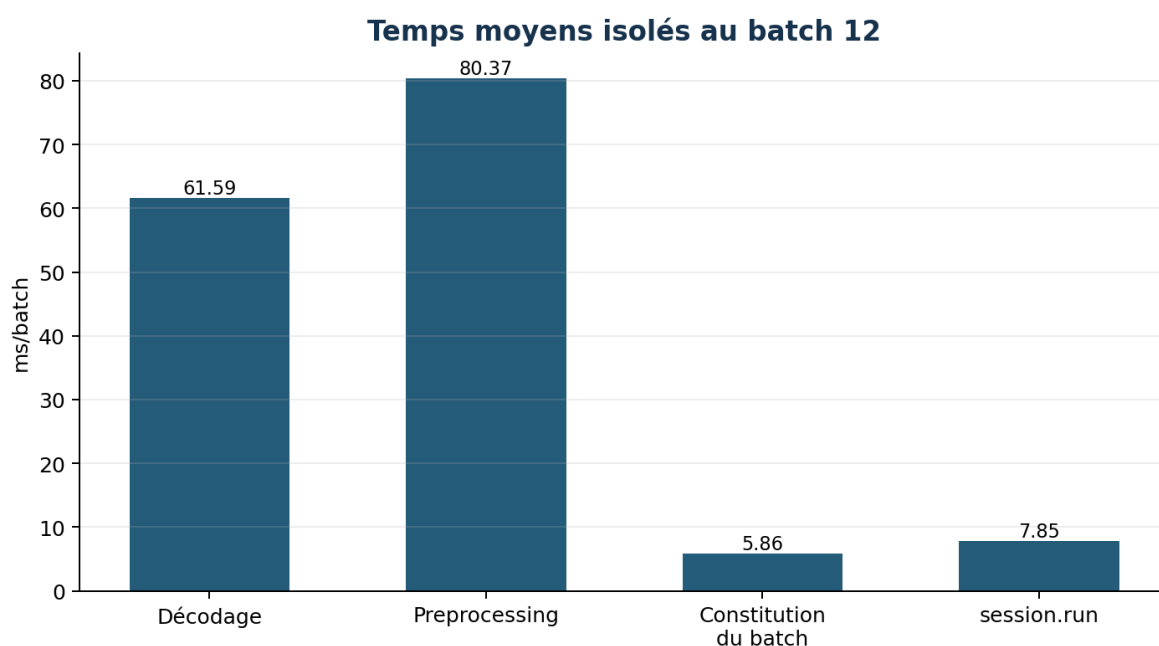


Figure 9 - Répartition des temps isolés au batch 12. Source : batch_benchmark.json

L'inférence ONNX Runtime représente moins de 8 ms pour douze images, tandis que décodage et preprocessing dépassent 140 ms dans les mesures isolées. Le principal goulot observé est donc la préparation CPU. Les valeurs isolées ne sont pas additives à l'end-to-end, car elles proviennent de boucles distinctes avec des allocations et états de cache différents.

Conséquence - Remplacer ResNet18 par un modèle plus petit ne garantit pas une amélioration importante du pipeline complet. L'optimisation du décodage et du preprocessing est prioritaire.

3.4 Discussion de la cible GTX 1060 3 Go

Le fichier ResNet18 ONNX occupe seulement 42,63 Mio, ce qui indique que les poids seuls sont modestes devant 3 Go. Cette taille ne suffit toutefois pas à valider la mémoire d'exécution : activations, buffers, runtime CUDA, pipeline d'images et autres processus consomment également de la VRAM.

Tableau 13 - Ce qui est validé et ce qui reste à valider sur la cible

Dimension	État	Conclusion autorisée
Fichier ONNX	42,63 Mio mesurés	Poids stockables ; mémoire runtime inconnue
VRAM	Non mesurée par processus	Impossible d'affirmer que 3 Go suffisent
Latence GPU	RTX 3080 Laptop mesurée	Aucune latence GTX 1060 déductible
CPU	CPU actuel mesuré	Ancien i7 non validé
Douze flux	Calcul et batching synthétique	Pas de validation réelle concurrente
Système	Windows 11	Résultats dépendants de l'environnement mesuré

Source : benchmark_results.json et limites expérimentales

Limite essentielle - Les mesures démontrent la faisabilité sur la machine de développement et identifient les goulots du pipeline, mais elles ne constituent pas une validation directe de la GTX 1060 3 Go.

3.5 Limites et recul critique

- Le matériel cible n'était pas disponible ; la RTX 3080 Laptop est beaucoup plus récente que la GTX 1060.
- La VRAM propre au processus n'a pas été mesurée de manière fiable.
- Les entrées sont des fichiers image ; la capture et le décodage vidéo réels de douze flux ne sont pas reproduits intégralement.
- La concurrence, les files d'attente, l'ordonnancement et la contention CPU/GPU d'une application complète ne sont pas simulés.
- Le batching dynamique mesure une mutualisation potentielle, pas un système temps réel à douze caméras.
- Les résultats sont liés à Windows, aux versions d'ONNX Runtime et aux providers installés.
- Aucune extrapolation numérique n'est réalisée de la RTX 3080 vers la GTX 1060.

Nuances relevées lors de la consolidation :

- Les mesures batch 1 statique et batch 1 dynamique proviennent de campagnes distinctes; leurs faibles écarts ne doivent pas être fusionnés.
- Les mesures isolées des étapes et la mesure end-to-end reposent sur des boucles différentes; elles ne sont pas strictement additives.
- La VRAM propre au processus est absente des artefacts et ne peut pas être chiffrée.

Points forts du protocole

- Exports ONNX contrôlés par ONNX checker puis comparés numériquement à PyTorch.
- Mesures CPU et CUDA après warm-up, sur plusieurs centaines d'itérations avec moyenne, médiane et percentiles.
- Séparation entre inférence seule et chaîne end-to-end.
- Comparaison de trois architectures réellement entraînées et exportées.
- Validation du batch dynamique pour N = 1, 4 et 12, puis benchmark jusqu'au batch 12.
- Séparation explicite entre mesures réelles et analyse qualitative de la cible.

3.6 Pistes d'optimisation

Les propositions suivantes sont des travaux futurs ; aucune n'a été mise en œuvre dans le cadre du §4.3.

Tableau 14 - Priorités d'optimisation futures

Priorité	Piste	Motivation
1	Optimiser décodage et preprocessing	Goulot dominant observé
2	Paralléliser le preprocessing CPU	Alimenter le GPU sans attente
3	Pipeline asynchrone et files d'attente	Découpler capture, préparation et inférence
4	Prétraitement GPU si pertinent	Réduire la charge CPU et les conversions
5	Batching dynamique piloté par délai	Mutualiser sans dégrader excessivement la latence
6	Benchmark direct GTX 1060 et mesure VRAM	Seule validation fiable de la cible
7	TensorRT, FP16 puis quantification	À mesurer sur la cible, sans gain présumé
8	Échantillonnage adaptatif des frames	Éviter plusieurs inférences sur la même châtaigne

Source : Interprétation des benchmarks §4.3

3.7 Conclusion du §4.3

ResNet18 est rapide en inférence CUDA sur la RTX 3080 Laptop mesurée et son export ONNX est fidèle au modèle PyTorch. La comparaison ne révèle aucune raison immédiate de le remplacer par EfficientNet-B0 ou MobileNetV3 Small dans ce contexte GPU ; MobileNet demeure cependant nettement plus intéressant sur CPU.

La conclusion principale porte sur le système complet : le batching améliore fortement l'utilisation du GPU, mais le débit end-to-end reste limité par le décodage et le preprocessing CPU. Les scénarios de douze caméras à 1 ou 2 FPS sont couverts théoriquement sur la machine actuelle ; le scénario de douze caméras à 5 FPS reste légèrement hors de portée et les flux réels n'ont pas été simulés.

Arrêt raisonné des expériences - L'étude dispose désormais d'un export vérifié, de mesures CPU/CUDA et end-to-end, d'une comparaison de trois architectures, d'un batch dynamique jusqu'à 12 et d'une identification du goulot. Sans le matériel cible, poursuivre apporterait peu de valeur et encouragerait des extrapolations fragiles. La prochaine expérience informative est un benchmark direct sur la GTX 1060 / i7 cible.

Références

- analysis\output\modeling\resnet18\onnx\export_summary.json
- analysis\output\modeling\resnet18\onnx\numerical_validation_details.csv
- analysis\output\modeling\resnet18\onnx\benchmark_smoke.json
- analysis\output\modeling\resnet18\onnx\benchmark_results.json
- analysis\output\modeling\onnx_comparison\onnx_comparison.json
- analysis\output\modeling\onnx_comparison\onnx_comparison.csv
- analysis\output\modeling\onnx_comparison\onnx_comparison_report.md
- analysis\output\modeling\resnet18\onnx\batch_benchmark\dynamic_export_summary.json
- analysis\output\modeling\resnet18\onnx\batch_benchmark\batch_benchmark.json
- analysis\output\modeling\resnet18\onnx\batch_benchmark\batch_benchmark.csv
- analysis\output\modeling\resnet18\onnx\batch_benchmark\batch_benchmark_report.md
- analysis\output\modeling\resnet18\final_test\final_test_metrics.json
- analysis\output\modeling\resnet18\onnx\resnet18_castagnet.onnx
- analysis\output\modeling\resnet18\onnx\resnet18_castagnet_dynamic_batch.onnx
- training\export_onnx.py
- training\benchmark_onnx.py
- training\benchmark_dynamic_batch.py
- training\compare_onnx.py